Topic

"DevOps BasedAutomatedDeployment System"



Field Project Submitted to

M.S.P. Mandal's

Deogiri College

ChhatrapatiSambhajinagar,Maharashtra

(An AutonomousCollegeAffiliatedtoDr.BabasahebAmbedkarMarathwadaUniversity)

For the partial fulfillment of B.C.A. F. Y.

Under the Faculty of Commerce & Management

By

Student Name: Kaveri Sanjay pathe
Class: B.C.A Third Year
PRN No.: 2425003636

Under the Guidance

(Samik shaBari)

Assistant Professor and Research Guide Department of Commerce and Management Deogiri College, Chhatrapati Sambhajinagar (MH)

M.S.P. Mandal's

Deogiri College

ChhatrapatiSambhajinagar,Maharashtra

(An AutonomousCollegeAffiliatedtoDr.BabasahebAmbedkarMarathwadaUniversity)

Certificate

This is to certify that present thesis entitled **"DevOps Based Automated Deployment system "** is a bona-fide work prepared by ***kaveri sanjay pathe*** under my guidance and supervision for the partial fulfillment of B.C.A. Third year in the Faculty of Commerce & Management, Deogiri College, Chhatrapati Sambhajinagar. He has been fulfilled the requirement for the submission of this field project to the best of my knowledge and not previously submitted to this or any other college / university for the award of any degree and diploma.

Project Guide

(SamikshaBari)
Assistant professor & Research Guide

Coordinator

Dr.P.L.ChitnisProf. Rajesh B. Lahane

Head of Department

Department of Commerce and Management

Place: Chhatrapati Sambhajinagar

Date: 24/10/2026

Declaration

I hereby declare that the present thesis entitled **"DevOps Based Automated Deployment System "** submitted by me for the Partial fulfillment of B.C.A. Third year in the faculty of Commerce and Management Deogiri College, Chhatrapati Sambhajinagar. is bona-fide work prepared by me under the guidance of **(Samiksha Bari)** further declare that, to the best of my knowledge and the same has not been submitted before this for any degree or diploma to any college / university / institute.

Place: Chhatrapati Sambhajinagar
Date: 24/10/2026

Name of Student: kaveri sanjay pathe
B.C.A. Third Year

Acknowledgement

First of all, I express my deep sense of gratitude to my respected guide
Dr. Samiksha Bari who has very kindly guided me and provided invaluable suggestion along with all kind of intellectual guidance from time to time. He also helped continuous encouragement, precious guidance and supervision for the success of this work. Otherwise, it would not have been possible me to complete this work.

I am extremely grateful to **Prof. Ravi Patil** (Principal) and **Dr. Ganesh Mohite,** (Vice Principal) of Deogiri College for their encouragement. I am also indebted to **Prof. Rajesh B. Lahane**, Head, Department of Commerce and Management and **P. L. Chitnis**, Coordinator of BBA and BCA**. Ms. Samiksha Bari, Mr. Sankesh More, Ms.Poonam Relan** all teaching and non-teaching staff of Department of Commerce and management, Deogiri College, Chhatrapati Sambhajinagar for their proper motivation to complete this research work.
I express my deep sense of gratitude to my mother, father, and all my family members who helped me to complete this research work.
Finally I record my sincere gratefulness to all those who helped me directly or indirectly to complete this research work.

(Kaveri Sanjay pathe)

Contents

Chapters	Title	Page No.
	Title Page	i
	Certificate	ii
	Declaration	iii
	Acknowledgement	iv
	Contents	v
Chapter I	Introduction and Research Methodology	
Chapter II	Review of literature	
Chapter III	Profile of Organisation or theme	
Chapter IV	Data Collection and Analysis	
Chapter V	Summary, Conclusions and Suggestion.	
	Bibliography	
	Annexure	

CHAPTER : 1
INTRODUCTION AND RESEARCH METHODOLOGY

CHAPTER I — INTRODUCTION AND RESEARCH METHODOLOGY

1.1 Introduction

Modern applications are updated frequently to introduce features, resolve defects, and address security issues. A release process that depends entirely on manual commands may vary between operators or environments. A forgotten step, incorrect configuration, or wrong artifact can cause downtime or make a release difficult to reproduce. As the frequency of releases increases, teams need a controlled and repeatable delivery process. DevOps combines people, processes, and automation to improve collaboration between development and operations. Continuous integration encourages frequent validation of changes, while continuous delivery makes verified software available for controlled release. Containers package the application and its dependencies. Kubernetes manages container workloads and helps maintain the desired application state. Cloud services provide scalable infrastructure and managed capabilities.

1.2 Problem Statement

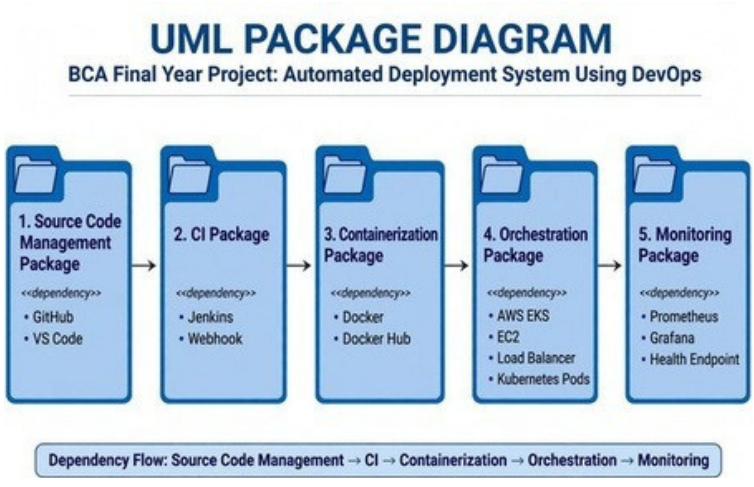
The problem addressed by this project is the lack of a standardised, auditable, and observable application deployment workflow. Manual deployment can result in inconsistent builds, unclear version history, late discovery of defects, and limited evidence about why a release failed. A suitable system should automate repeatable tasks, stop when quality checks fail, deploy a known image, verify service health, and provide a documented recovery method.

1.3 Aim

The aim is to design and document an automated deployment and monitoring system that uses a CI/CD pipeline, Docker, Kubernetes, and AWS to deliver an application in a controlled and observable manner.

1.4 Objectives

- Automate source checkout, validation, testing, image build, and release steps.
- Package application files and runtime dependencies into a Docker image.
- Store versioned images in a registry such as Amazon ECR.
- Deploy the selected image to Kubernetes and verify the rollout.
- Collect service and infrastructure metrics and present them in dashboards.
- Define security controls, test cases, failure handling, and rollback procedures.



SCOPE, METHODOLOGY AND FEASIBILITY

1.5 Scope of the Project

The scope includes a small web application or sample service, source control, automated validation, container image creation, registry publication, Kubernetes deployment, service exposure, monitoring, and operational documentation. Jenkins is used as a representative CI/CD tool, Docker for packaging, Amazon ECR for image storage, Amazon EKS for managed Kubernetes, and Prometheus with Grafana for observability. Equivalent tools can be substituted when required by the institution or available infrastructure.

The project is an academic reference design rather than a claim that a production environment has already been deployed. Production readiness would require detailed capacity planning, security review, compliance assessment, backup strategy, disaster recovery, service-level objectives, and budget approval. A local cluster can be used to validate manifests before creating cloud resources.

1.6 Research and Development Methodology

The project follows an iterative engineering methodology. First, the deployment problem and stakeholder needs are identified. Second, functional and non-functional requirements are documented. Third, the architecture is divided into source control, pipeline, container registry, orchestration, cloud networking, and monitoring components. Each component is tested independently and then integrated into an end-to-end workflow.

- Requirement analysis: identify users, inputs, outputs, constraints, and acceptance criteria.
- Design: define the architecture, release stages, trust boundaries, and failure paths.
- Implementation: prepare pipeline definitions, container configuration, and Kubernetes manifests.
- Verification: test successful and unsuccessful builds, rollout behaviour, and monitoring.
- Evaluation: record observed results and compare the automated process with the previous workflow.

1.7 Feasibility Study

Technical feasibility is supported by established tools and documented integration patterns. Operational feasibility depends on the team's ability to manage repositories, CI agents, container images, Kubernetes resources, and AWS permissions. Economic feasibility depends on cloud resource size and duration. Managed clusters, worker nodes, load balancers, NAT gateways, storage, and data transfer may incur charges; therefore, the implementation should use a budget and cleanup plan.

1.8 Assumptions and Limitations

The design assumes access to a repository, CI runner, AWS account or approved Kubernetes target, image registry, and the permissions needed for deployment. Credentials are represented by placeholders. The actual application language, health endpoint, port, cluster version, and resource settings must be adapted to the selected application.

DEVOPS PRINCIPLES AND PROJECT PLAN

1.9DevOps principles

DevOps emphasises shared ownership, small and frequent changes, automation, measurable outcomes, and continuous feedback. It does not consist of a single product. Teams use automation to make repeatable actions consistent, and they use monitoring and incident reviews to improve future changes. Collaboration between developers and operators helps make applications easier to deploy and maintain.

1.10 Continuous Integration

Continuous Integration (CI) is the practice of merging changes frequently and validating them automatically. A CI pipeline may run formatting checks, unit tests, static analysis, dependency checks, and packaging. Quick feedback helps developers identify defects close to the change that introduced them. Required checks should prevent failed changes from progressing to release stages.

1.11 Continuous Delivery and Continuous Deployment

Continuous Delivery keeps software in a releasable state after automated checks have passed. A release may require an authorised approval before production. Continuous Deployment automatically releases changes that pass all required controls. The appropriate choice depends on the application's risk, organisational policy, and testing maturity.

1.12 Project Plan

Phase	Activity	Deliverable
1	Requirement analysis and research	Requirement specification
2	Architecture and tool selection	System architecture
3	Application packaging	Dockerfile and image
4	CI pipeline configuration	Automated test/build pipeline
5	AWS and Kubernetes integration	Deployment resources
6	Monitoring and verification	Dashboard and test evidence
7	Documentation	Final report and runbook

1.13 Expected Deliverables

The deliverables are a source repository, a pipeline definition, a Dockerfile, versioned container image, Kubernetes Deployment and Service manifests, monitoring configuration, a test-case table, and a short operational runbook. Screenshots, execution times, and test outcomes should be added after actual execution.

REVIEW OF LITERATURE AND EXISTING SYSTEMS

2.1 Existing Manual Deployment

In a manual workflow, an operator may download or pull source code, install dependencies, build the application, copy files to a server, restart the service, and verify the result. This approach can be understandable for a small project, but it becomes error-prone when the steps are repeated across environments. Different operators may use different commands, and the exact version deployed may not be clearly recorded.

2.2 Automated Deployment Practices

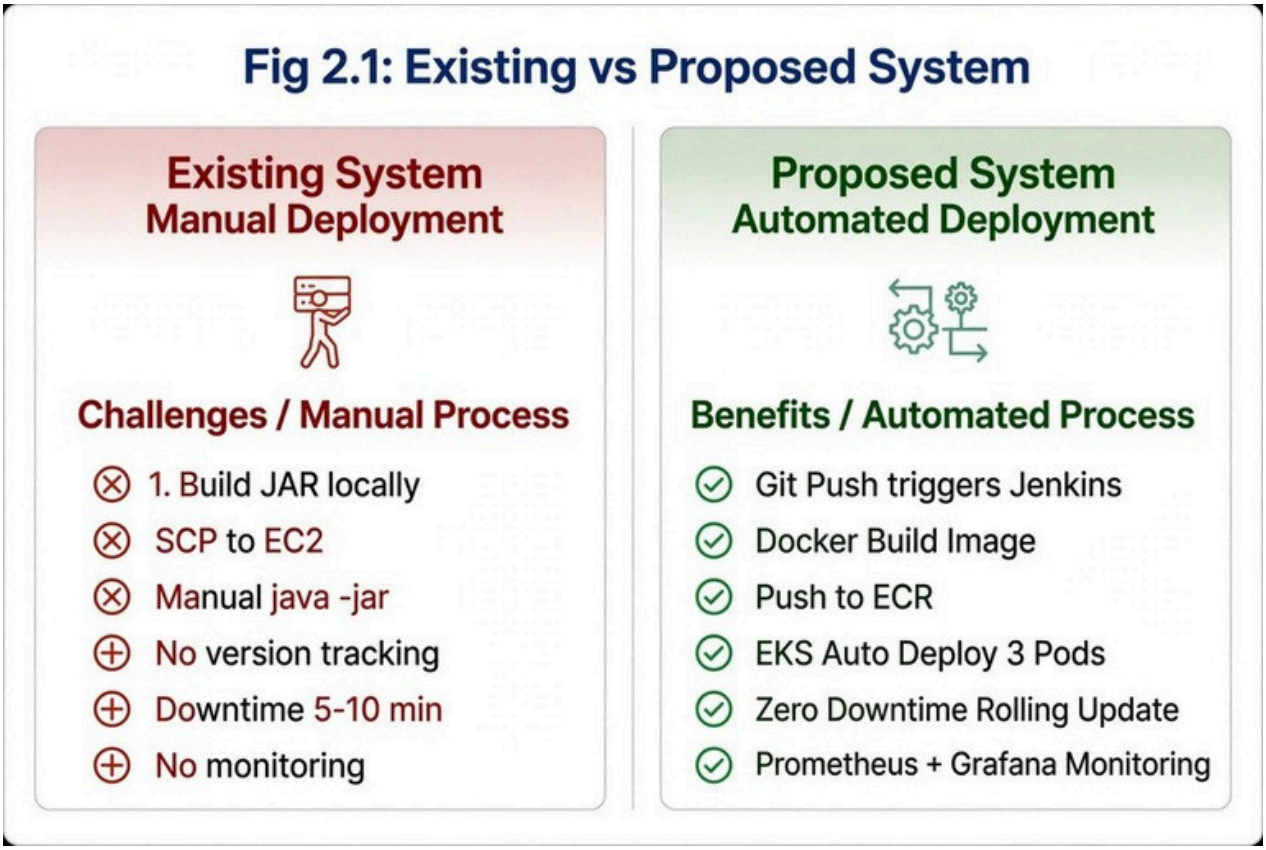
Modern deployment practices use pipeline-as-code, automated tests, immutable artifacts, declarative infrastructure, and deployment health checks. The pipeline becomes a repeatable record of how a release was produced. Declarative manifests describe the desired workload state, and the orchestrator works to converge the running environment toward that state.

2.3 Git and Version Control

Git records changes as commits and supports branching, review, and recovery to earlier revisions. A commit identifier can be associated with an image tag, helping an operator connect a running release to its source revision. Protected branches, code review, and required checks reduce the risk of unreviewed changes entering a release branch.

2.4 Jenkins and Pipeline Automation

Jenkins is an automation server that can execute pipelines described as code. A pipeline coordinates checkout, testing, packaging, publishing, and deployment. Its value depends on reliable tests, maintained agents, secure credentials, and controlled permissions. Pipeline definitions should be reviewed because an unsafe pipeline can expose secrets or alter production resources.



CHAPTER : 2
DOCKER KUBERNETES And AWS

CHAPTER II — DOCKER, KUBERNETES AND AWS

2.5Docker Containerisation

A container image packages application files and runtime dependencies in a standard deployment
environments, making it easier to test the same artifact through multiple stages. A Dockerfile describes how the image is built. Good practices include using maintained base images, minimising unnecessary packages, excluding secrets, and running as a non-root user where practical.

2.6 Kubernetes Orchestration

Kubernetes manages containerised workloads through declarative resources. A Deployment defines a desired number of replicas and controls rolling updates. Pods run the containers, while a Service provides a stable network endpoint. Readiness probes indicate whether a container can receive traffic; liveness probes can restart containers that are stuck or unhealthy. Resource requests and limits help control scheduling and consumption.

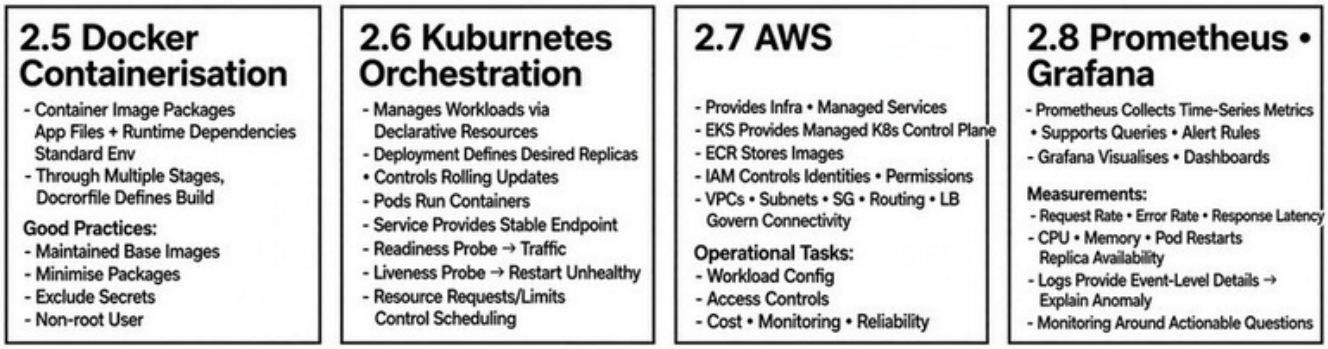
2.7 Amazon Web Services

AWS provides infrastructure and managed services for hosting applications. Amazon EKS provides a managed Kubernetes control plane, and Amazon ECR stores container images. IAM controls identities and permissions. VPCs, subnets, security groups, routing, and load balancers govern network connectivity. Although managed services reduce some operational tasks, users remain responsible for workload configuration, access controls, cost monitoring, and many reliability decisions.

2.8 Prometheus and Grafana

Prometheus collects time-series metrics from configured endpoints and supports queries and alert rules. Grafana visualises metrics in dashboards. Useful measurements include request rate, error rate, response latency, CPU and memory consumption, Pod restarts, and replica availability. Logs provide event-level details that help explain a metric anomaly. Monitoring should be designed around actionable questions rather than collecting every possible metric.

Fig 2.2: Docker, Kuburnetes and AWS Stack



COMPARISON AND RESEARCH GAP

2.9 Manual and AutomatedSystemComparison

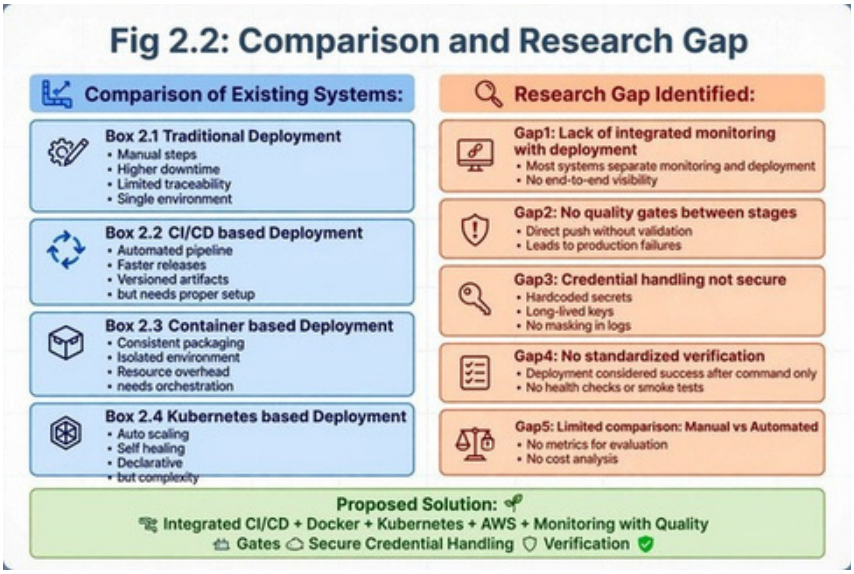
Aspect	Manual approach	Proposed automated approach
Build	Commands run by a person	Repeatable pipeline stage
Testing	May be inconsistent	Automated checks gate release
Artifact	Can be rebuilt differently	Versioned container image
Deployment	Manual server or cluster changes	Declarative Kubernetes rollout
Traceability	Depends on manual notes	Commit, image tag and pipeline history
Monitoring		Metrics, dashboards and alert rules
Recovery		Documented rollback and verification

2.10 Benefits Identified from the Review

Automation can reduce repetitive steps, standardise build and release procedures, and make results easier to audit. Containers provide a consistent package, Kubernetes supports controlled rollout and desired-state management, and monitoring gives feedback after deployment. These benefits are conditional: poor tests, insecure permissions, or incorrect probes can still produce an unreliable release.

2.11 Research Gap

A demonstration that only builds an image and deploys it does not fully address operational reliability. A complete learning project should also include failure handling, traceability, readiness checks, monitoring, least-privilege access, cost awareness, and rollback planning. This project combines these topics in a single workflow so that delivery automation and operational feedback are considered together.



CHAPTER : 3
REQUIREMENTS ANALYSIS

CHAPTER III — REQUIREMENTS ANALYSIS

3.1 System Overview

The system is a reference platform for deploying a web application. Developers submit changes to a repository, the pipeline validates and packages those changes, the registry stores a versioned image, Kubernetes runs the image, and monitoring reports service behaviour. The primary users are developers, DevOps engineers, and system administrators. **3.2 Functional Requirements**

- FR1: Store application code and deployment configuration in version control.
- FR2: Trigger automated checks when code is pushed or a pull request is opened.
- FR3: Run tests and stop the pipeline if required checks fail.
- FR4: Build a Docker image with a traceable version tag.
- FR5: Publish the image to an authorised container registry.
- FR6: Deploy the image to Kubernetes and verify rollout status.
- FR7: Collect metrics and present operational dashboards.
- FR8: Record pipeline results and document rollback steps.

3.3 Non-Functional Requirements

- Reliability: failed deployment states should be detected and recoverable.
- Security: use least privilege, protected secrets, and trusted image sources.
- Maintainability: store pipeline and infrastructure configuration in version control.
- Scalability: allow replica counts and resource settings to be adjusted.
- Observability: expose useful health, performance, and deployment information.
- Portability: package the application in a compatible container image.

3.4 User Roles

Developers are responsible for code changes and unit tests. Reviewers approve changes and ensure required checks pass. The CI service executes automated stages using a restricted identity. Cluster administrators maintain cluster-level access and policies. Operators monitor service health, respond to alerts, and follow the incident runbook. Roles should be separated so that routine build access does not automatically grant broad administrative privileges.

HARDWARE, SOFTWARE AND ARCHITECTURE

3.5Hardware Requirements

A modern development computer with a multi-core processor, 8 GB RAM minimum and 16 GB recommended for local containers, adequate disk space, and a reliable internet connection is suitable for development. If builds and Kubernetes workloads run in cloud infrastructure, local hardware requirements may be lower. Actual requirements depend on application size and the number of concurrent tasks.

3.6 Software Requirements

Component	Purpose	Example
Version control	Track source and configuration	Git
CI/CD	Coordinate build and release stages	Jenkins Pipeline
Container build	Package application dependencies	Docker
Registry	Store versioned images	Amazon ECR
Orchestration	Run and update containers	Kubernetes on EKS
Metrics	Collect time-series measurements	Prometheus
Dashboards	Visualise metrics and alerts	Grafana
Cloud platform	Networking, identity and compute	AWS IAM, VPC, EKS

3.7 High-Level Architecture

The main path is Developer → Git Repository → CI Pipeline → Validation and Tests Docke→r
→

Image → Amazon ECR → Kubernetes on EKS → Service or Load Balancer

Users. The monitoring path runs alongside this delivery path: application and cluster metrics are collected by Prometheus, visualised by Grafana, and used to generate alerts. Logs and platform events support investigation.

3.8 Component Responsibilities

- Git repository: stores source code, pipeline definitions, Dockerfile, and manifests.
- CI server: checks out code, runs tests, builds images, and records pipeline results.
- Container registry: stores images and controls push/pull access.
- Kubernetes cluster: schedules Pods, maintains replicas, and performs rolling updates.
- Service and load balancer: provide a stable network endpoint.
- Monitoring stack: collects metrics, visualises trends, and supports alerting.

PIPELINEDESIGN AND DATA FLOW

3 .9PipelineStages

Stage	Action	Failure handling
Checkout	Fetch source revision	Stop if repository access fails
Validate	Lint and validate configuration	Reject invalid code/configuration
Test	Run unit and integration tests	Do not publish a failed build
Build	Create container image	Save logs for investigation
Publish	Push image to ECR	Do not deploy an unpublished image
Deploy	Update Kubernetes workload	Stop and inspect failed rollout
Verify	Check rollout and health endpoint	Alert or roll back if necessary

3.10 Trigger and Promotion Strategy

Pull requests should run validation and tests without automatically changing production. A merge to the main branch may build and publish a candidate image. Deployment to a test environment can be automatic, while production can require authorised approval. The same immutable image should be promoted between environments rather than rebuilding different artifacts for each stage.

3.11 Artifact Traceability

The commit identifier should be included in image metadata or its tag. The pipeline should record the repository revision, build result, image reference, and deployment target. When an incident occurs, this information helps determine which source revision is running and which pipeline created it. Immutable tags or image digests reduce ambiguity caused by mutable labels.

3.12 Exception Flow

If tests fail, the pipeline stops before image publication. If registry authentication fails, deployment must not proceed. If Kubernetes cannot pull the image or Pods fail readiness checks, rollout verification reports the problem. The previous healthy version may remain available depending on the deployment strategy. Operators investigate logs and events, correct the issue, or roll back.

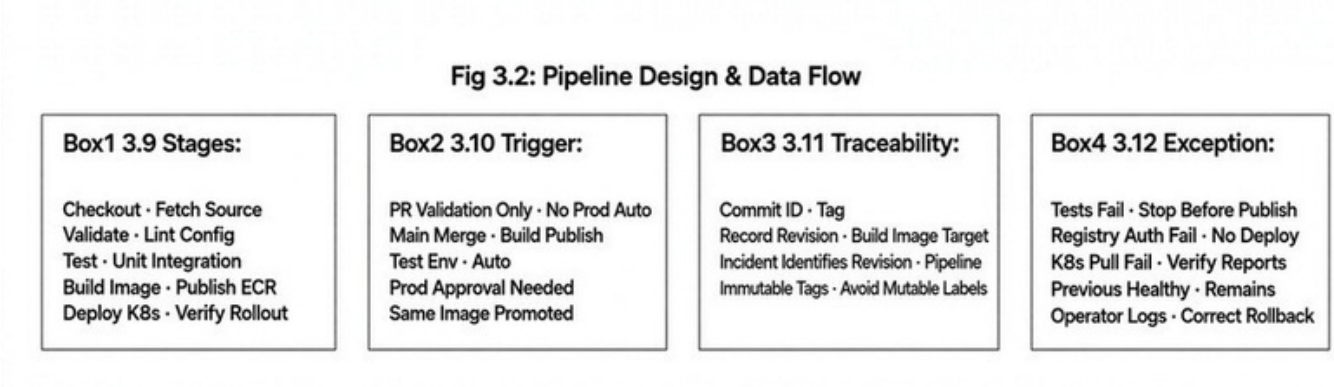


Fig 3.2: Pipeline Design & Data Flow

SECURITY AND NETWORK DESIGN

3.13Security Principles

Security should be included throughout the delivery lifecycle. Use least-privilege IAM policies, short-lived credentials where possible, protected branches, reviewed pipeline changes, dependency checks, image scanning, and restricted network exposure. Secrets should be stored in an approved secret manager or CI credential store and injected at runtime. They should never be committed to source control or printed in pipeline logs.

- Restrict image push permissions to the build pipeline and pull permissions to workloads.
- Use unique image tags and consider image digests for stronger immutability.
- Run containers as non-root where supported by the application.
- Set resource requests and limits to reduce resource contention.
- Use TLS for external endpoints and restrict administrative access.
- Review audit logs and rotate credentials according to policy.

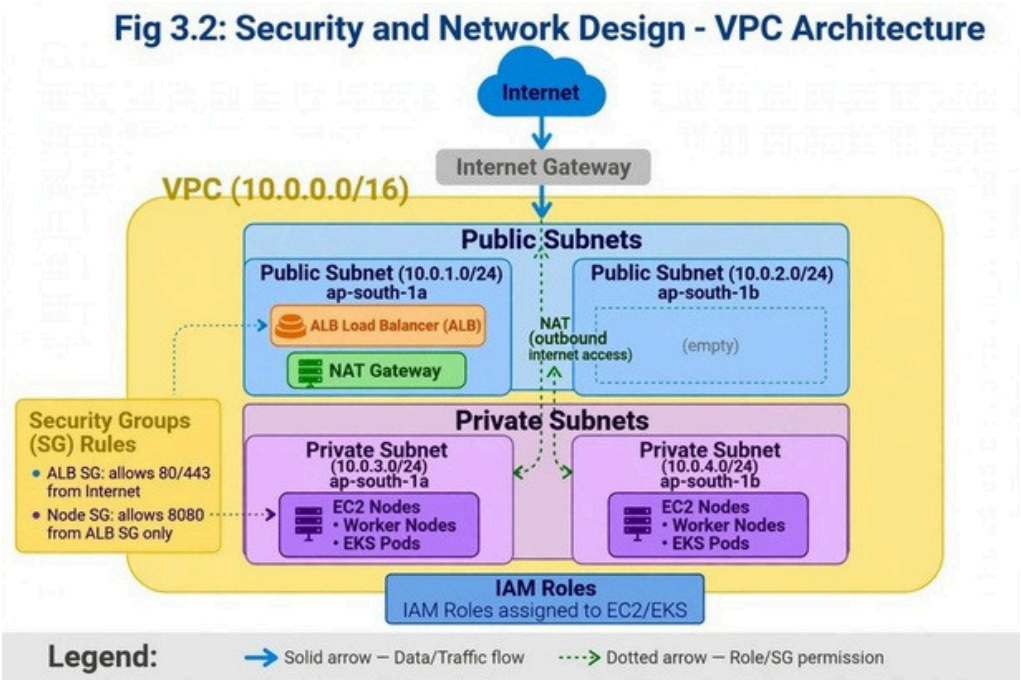
3.14 Network Design

An AWS deployment commonly uses a VPC with public and private subnets. Worker nodes and application Pods may be placed in private subnets, while a public load balancer exposes only the intended application endpoint. Security groups and network policies should permit only required communication. Administrative access should use approved secure channels and should not expose cluster management endpoints unnecessarily.

3.15 Backup and Recovery

Source code and configuration should be recoverable through version control. Container images should follow a retention policy. Stateful application data requires a separate backup and restore strategy because rolling back a container does not reverse database changes or restore deleted data. Recovery procedures should be tested and documented.

Fig 3.2: Security and Network Design - VPC Architecture



DESIGN DECISIONS AND FEASIBILITY

3.16Why Docker?

Docker provides a consistent application package that includes required runtime components. It reduces dependency differences between environments and supports repeatable testing. The image should contain only necessary files, use maintained base images, and avoid embedded credentials. Multi-stage builds can reduce the final image size when the application requires a compilation step.

3.17 Why Kubernetes?

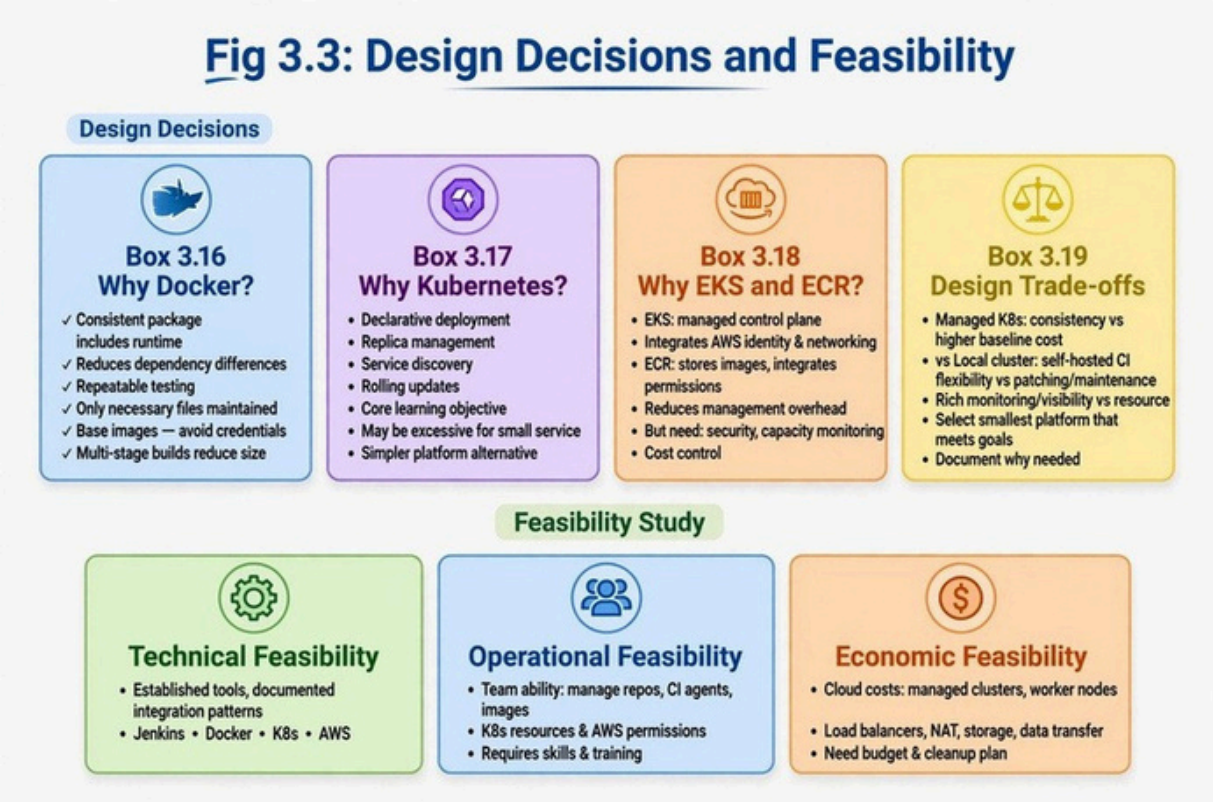
Kubernetes is useful when an application benefits from declarative deployment, replica management, service discovery, and rolling updates. It may be excessive for a very small service that could run on a simpler managed platform. The project chooses Kubernetes because orchestration and rollout behaviour are core learning objectives.

3.18 Why EKS and ECR?

EKS provides a managed Kubernetes control plane and integrates with AWS identity and networking. ECR stores images and integrates with AWS permissions. These services reduce some management tasks but do not eliminate the need for workload security, capacity planning, monitoring, and cost control.

3.19 Design Trade-offs

Managed Kubernetes improves consistency but can have a higher baseline cost than a local cluster. A self-hosted CI server offers flexibility but requires patching and maintenance. Rich monitoring increases visibility but adds resource and configuration overhead. The design should select the smallest platform that meets the project goals and document why each component is needed.



CHAPTER : 4 IMPLEMENTATION OVERVIEW

CHAPTER IV — IMPLEMENTATION OVERVIEW

4.1Repository Structure

A maintainable repository separates application source, tests, container configuration, pipeline code, Kubernetes manifests, and documentation. A suggested structure is: app/ for application code; tests/ for automated tests; Dockerfile for image build instructions; Jenkinsfile for pipeline stages; k8s/ for deployment and service resources; monitoring/ for dashboards and alerts; and README.md for setup and recovery instructions.

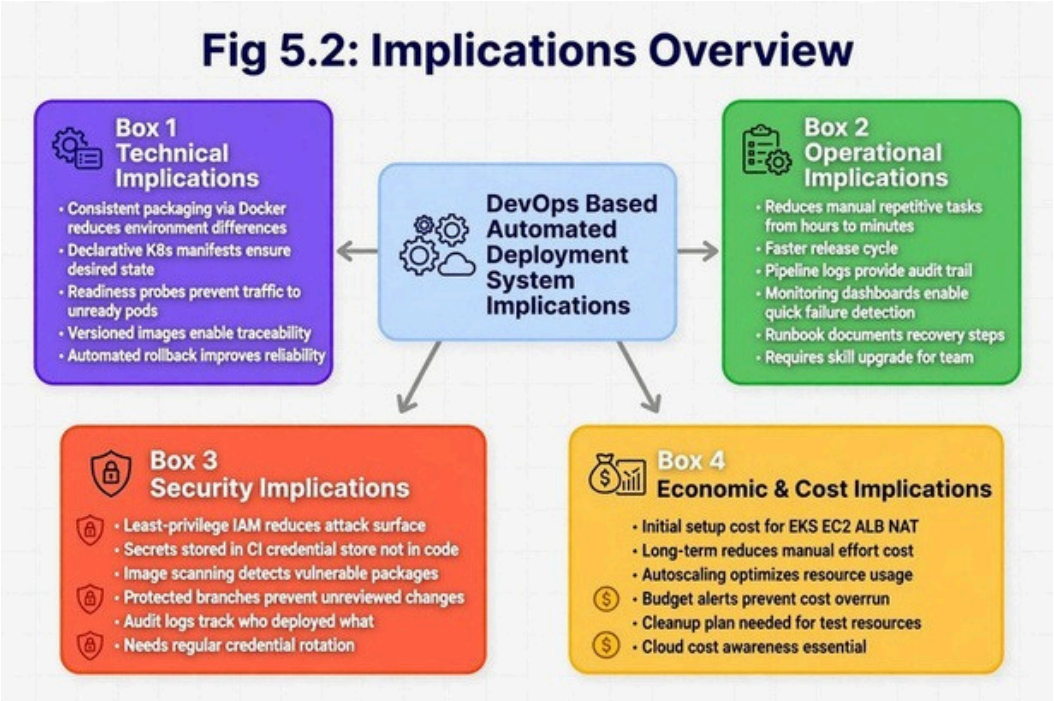
4.2 Source Control Workflow

Create a feature branch for each change, commit focused updates, and open a pull request for review. Automated checks should run on the pull request. Merge only after required checks pass and approvals are complete. Use descriptive commit messages and release tags. Generated files, local environment files, and secrets should be excluded from the repository. **4.3 Application Health Endpoint**

The application should expose a health endpoint that returns success when the service is ready to receive requests. The endpoint should not reveal environment variables, credentials, or internal infrastructure details. Logs should be written to standard output and standard error so container platforms can collect them. A readiness endpoint should reflect the ability to serve traffic, not merely that a process has started.

4.4 Configuration Management

Environment-specific settings should be kept separate from the application image. Non- sensitive values may be supplied through configuration resources, while secrets should be injected using a secure secret mechanism. Configuration changes should be reviewed and versioned. The same image can then be promoted through development, testing, and production with only approved environment configuration changing.



DOCKER IMAGE BUILD AND QUALITY

4.5 ContainerBuildProcess

The build process starts from a maintained base image, installs required dependencies, copies application files, and defines the startup command. A .dockerignore file excludes local dependencies, temporary output, version-control metadata, and environment files. Builds should be repeatable and should not depend on untracked local files.

4.6 Image Quality and Security

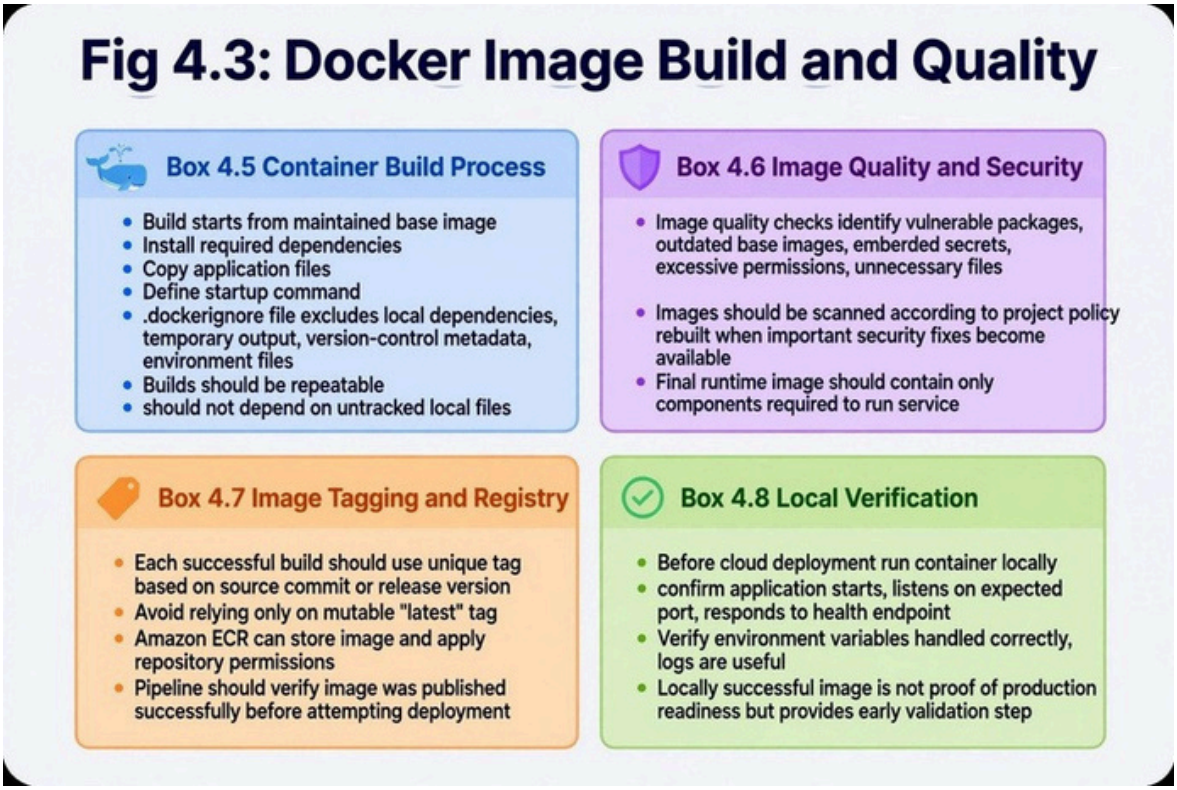
Image quality checks can identify vulnerable packages, outdated base images, embedded secrets, excessive permissions, and unnecessary files. Images should be scanned according to project policy and rebuilt when important security fixes become available. The final runtime image should contain only the components required to run the service.

4.7 Image Tagging and Registry

Each successful build should use a unique tag based on the source commit or release version. Avoid relying only on the mutable latest tag. Amazon ECR can store the image and apply repository permissions. The pipeline should verify that the image was published successfully before attempting deployment.

4.8 Local Verification

Before cloud deployment, run the container locally and confirm that the application starts, listens on the expected port, and responds to the health endpoint. Verify that environment variables are handled correctly and that logs are useful. A locally successful image is not proof of production readiness, but it provides an early validation step.



CI/CD PIPELINE IMPLEMENTATION

4.9 Pipeline Stage Details

The checkout stage retrieves the intended repository revision. The validation stage checks code formatting and configuration. The test stage executes unit and integration tests. The build stage creates the image using the selected Dockerfile. The publish stage authenticates to the registry and pushes a unique tag. The deploy stage updates the Kubernetes workload, and the verify stage confirms rollout and application health.

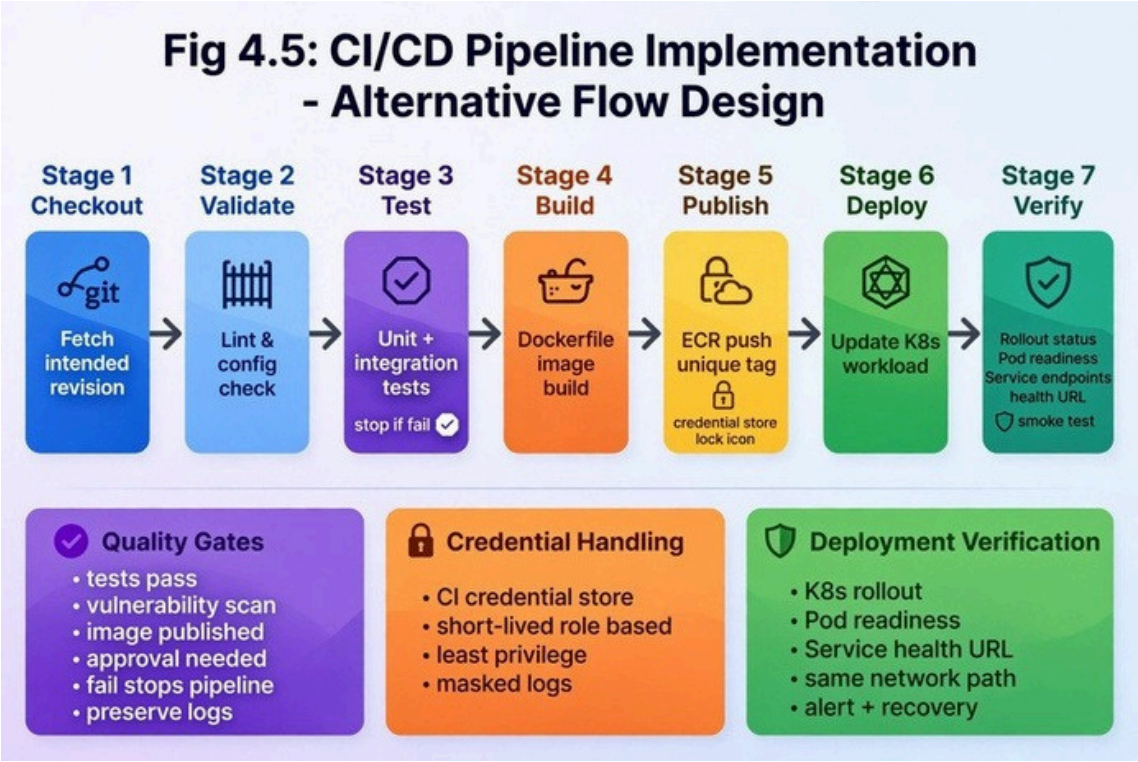
4.10 Pipeline Quality Gates

A quality gate defines a condition that must be satisfied before a release proceeds. Examples include successful tests, an acceptable vulnerability scan result, successful image publication, and an approved deployment. A failed stage should stop later release actions and preserve logs. The pipeline should report a clear reason for failure rather than merely returning a generic error.

4.11 Credential Handling

Credentials should be managed through a CI credential store or an approved cloud identity mechanism. Avoid long-lived access keys when short-lived role-based access is available. Permissions should be restricted to the required registry and cluster resources. Pipeline logs should be checked to ensure tokens and secret values are masked and never echoed. **4.12 Deployment Verification**

A successful command does not necessarily mean that users can access the application. Verification should check Kubernetes rollout status, Pod readiness, Service endpoints, and the health URL. Where possible, a smoke test should send a request through the same network path used by users. A failed verification should trigger an alert and a documented recovery decision.



AWS INFRASTRUCTURE SETUP

4.13 AWSResources

A typical deployment uses a VPC, subnets, route tables, security groups, IAM roles, an ECR repository, an EKS cluster, worker capacity, and a load balancer. Exact choices depend on region, application requirements, cluster version, availability goals, and budget. Use infrastructure-as-code or an approved repeatable setup process when possible.

4.14 Recommended Setup Sequence

• Choose an AWS region and estimate the cost before creating resources.

• Create or select a VPC and subnets for the cluster networking design.

• Create an ECR repository and configure authorised push and pull access.

• CreCaotnef aignu EreK CSI cpleursmteirs sainodn sa pfoprr popubrialisteh iwngo rikmear gceasp aac namespace.

• Configure authorised kubectl access for administrators and the deployment pipeline.

• Deploy the application, verify health, and only then enable external access.

4.15 IAM and Access Control

IAM roles and policies should grant only the actions required by each identity. The CI identity should not automatically have full account administrator permissions. Cluster access should
be restricted to approved users and workloads. Access should be reviewed periodically, and unused roles or credentials should be removed.

4.16 Cost Control and Cleanup

EKS control-plane charges, worker compute, NAT gateways, load balancers, storage, and
data transfer may create ongoing costs. Use a budget alert, small development resources,
and a planned cleanup process. After testing, inspect the account for leftover load balancers,
disks, addresses, and other resources. Verify billing rather than assuming that deleting one resource removes every charge.

KUBERNETES DEPLOYMENT AND SERVICE

4.17 DeploymentResource

A Kubernetes Deployment describes the desired replica count, container image, labels, ports, and update strategy. The selector must match the labels on the Pod template. The container port should match the application listener. Readiness probes prevent traffic from reaching a Pod before it is ready. Resource requests help the scheduler place workloads, while limits constrain consumption.

4.18 Service Resource

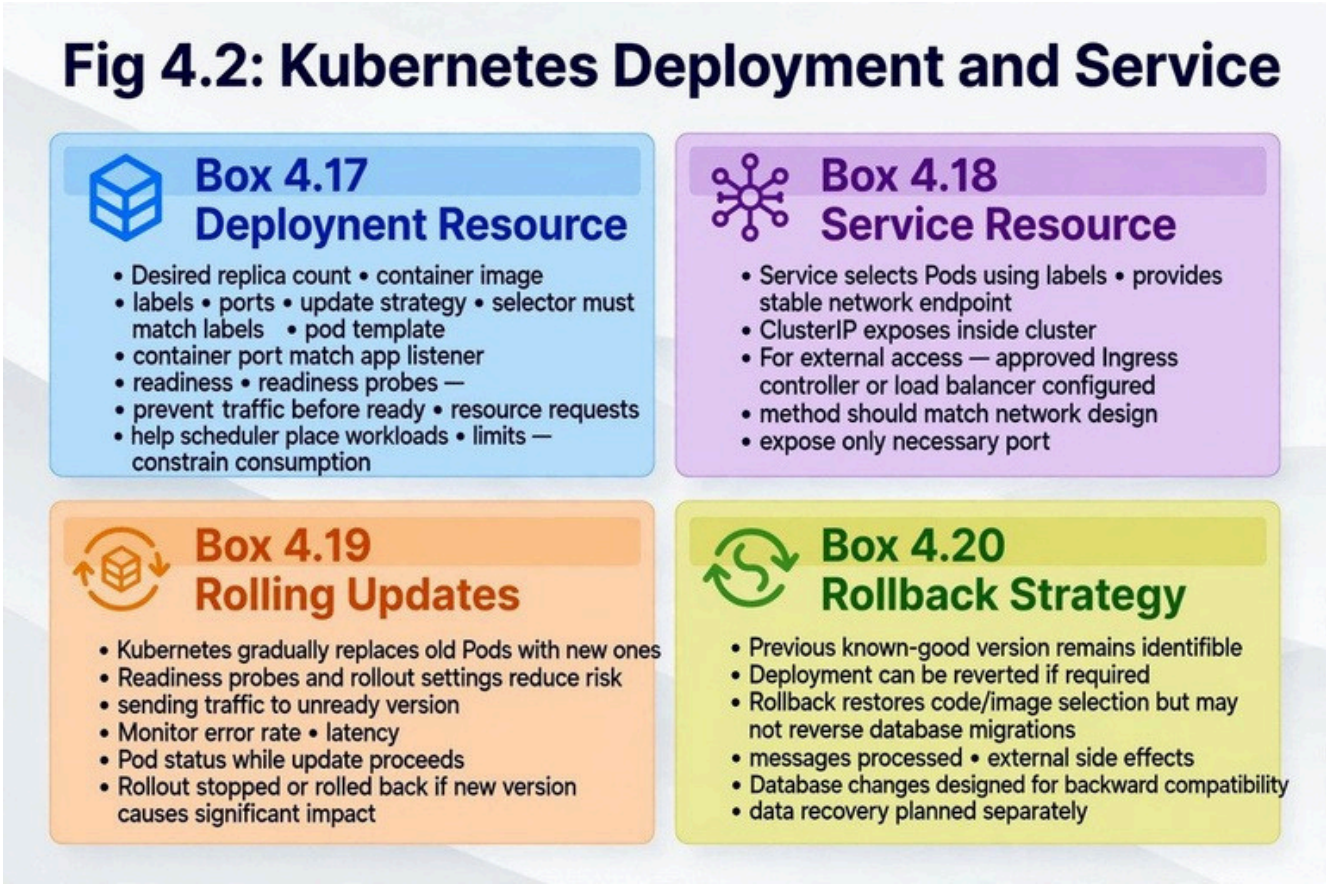
A Service selects Pods using labels and provides a stable network endpoint. A ClusterIP Service exposes the application inside the cluster. For external access, an approved Ingress controller or load balancer can be configured. The chosen method should match the network design and should expose only the necessary port.

4.19 Rolling Updates

During a rolling update, Kubernetes gradually replaces old Pods with new ones. Readiness probes and appropriate rollout settings reduce the risk of sending traffic to an unready version. Monitor error rate, latency, and Pod status while the update proceeds. A rollout should be stopped or rolled back if the new version causes significant impact.

4.20 Rollback Strategy

The previous known-good application version should remain identifiable so the Deployment can be reverted if required. Rollback restores application code and image selection, but it may not reverse database migrations, messages already processed, or external side effects. Database changes should be designed for backward compatibility and data recovery should be planned separately.



MONITORING AND OBSERVABILITY

4.21Monitoring Objectives

Monitoring should help answer whether the application is available, how it performs, and whether resources are nearing capacity. A basic dashboard can display request rate, error rate, response latency, CPU and memory usage, Pod restarts, and available replicas. Monitoring data should support operational decisions and be understandable to the people responsible for the service.

4.22 Prometheus Metrics Collection

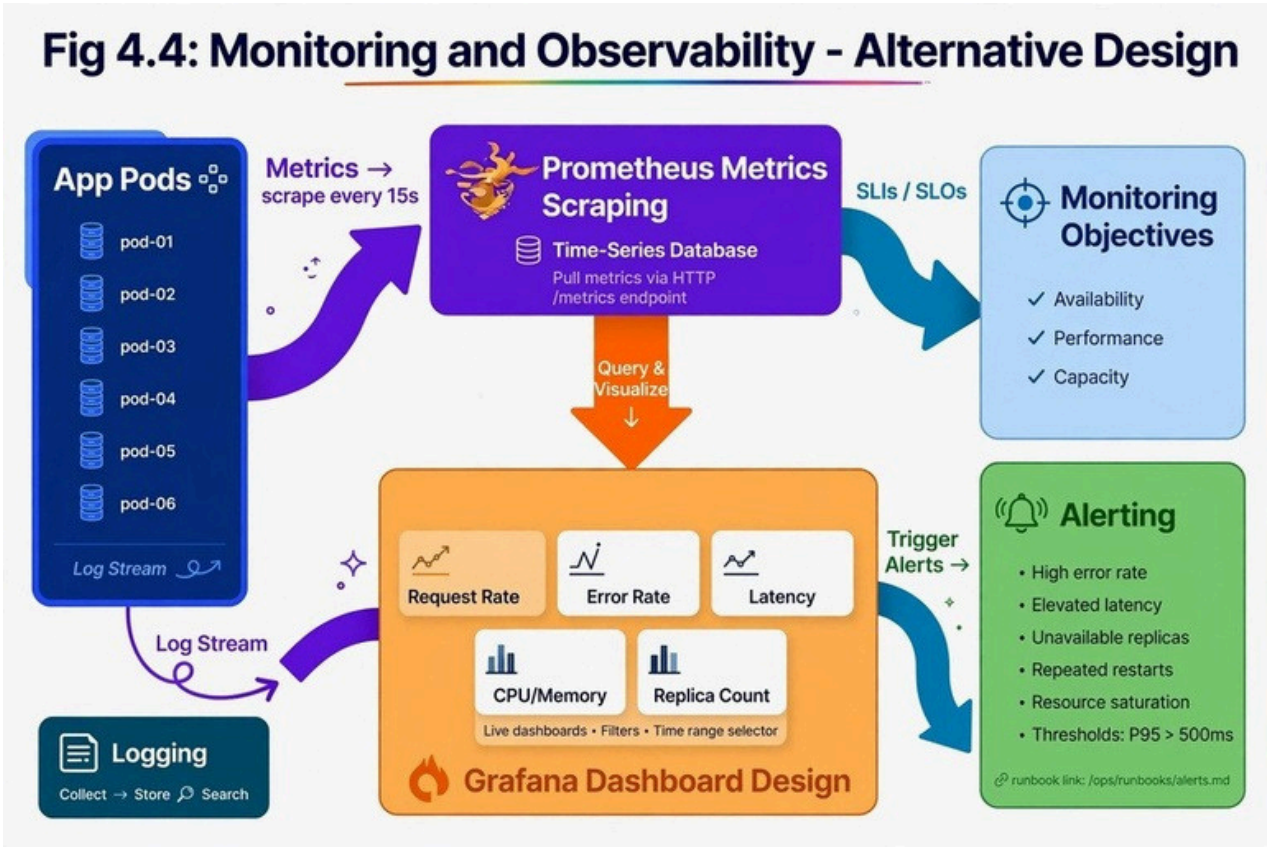
Prometheus scrapes metrics from configured endpoints at regular intervals and stores them as time series. Application metrics require an instrumented endpoint or exporter. Kubernetes and node metrics may require additional exporters and permissions. Metrics endpoints should be restricted where necessary, and collection intervals should balance useful resolution against system overhead.

4.23 Grafana Dashboard Design

A useful Grafana dashboard identifies the application and environment, uses clear units, and shows meaningful time ranges. Panels may include request throughput, error percentage, latency percentiles, CPU, memory, replica count, and restart count. Access should use role- based permissions. Operational dashboards should not be exposed publicly without a deliberate security review.

4.24 Alerting

Alerts should be based on conditions that require action. Examples include sustained high error rates, elevated latency, unavailable replicas, repeated restarts, and resource saturation. Thresholds should be selected from observed behaviour and service objectives. Too many low- value alerts can create fatigue, so alerts should include context, severity, and a runbook link where available.



LOGGING, TROUBLESHOOTING AND OPERATIONS

4.25 LoggingStrategy

Applications should emit useful logs to standard output and standard error. Each event can include a timestamp, severity, component name, and request or correlation identifier. Logs should not contain passwords, access tokens, personal data, or other secrets. Retention periods and access permissions should follow the organisation's requirements.

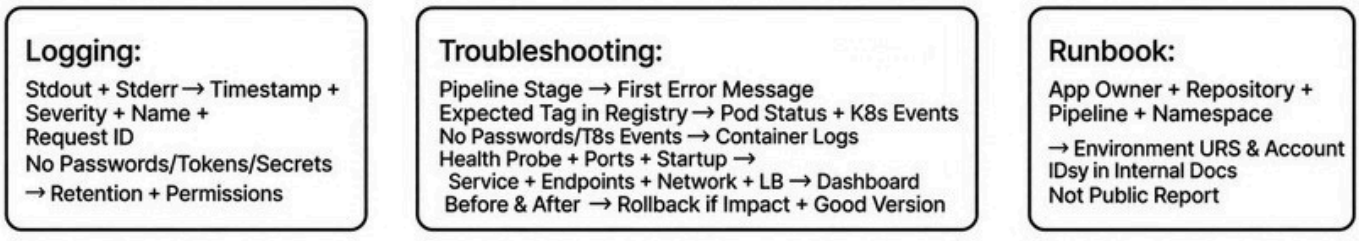
4.26 Troubleshooting Workflow

- Check the pipeline stage and identify the first meaningful error message.
- Confirm that the expected image tag exists in the registry.
- Inspect Pod status, Kubernetes events, and container logs.
- Check health-probe paths, ports, and startup timing.
- Verify Service selectors, endpoints, network policies, and load-balancer health.
- Compare dashboard metrics before and after the deployment.
- Roll back when user impact is significant and a known-good version is available.

4.27 Operational Runbook

The runbook should record the application owner, repository, pipeline location, namespace, Dsheopuloldy mednets ncarimbee , hheoawlth etnod poeinsct,a dlaatseh boaanr d, ianlceirdt ecnhta nn Environment-specific URLs and account identifiers should be stored only in approved internal documentation, not in a public academic report.

Fig 4.5: Logging, Troubleshooting and Operations



CHAPTER : 5
TESTING AND VALIDATION

CHAPTER V — TESTING AND VALIDATION

5.1TestingStrategy

Testing is performed at multiple levels. Unit tests validate individual functions, integration tests validate component interactions, container checks validate image configuration, and deployment tests confirm that Kubernetes reaches the desired state. Monitoring tests verify that metrics appear and alerts trigger under the intended conditions.

5.2 Test Cases

ID	Test	Expected result
TC-01	Commit valid code	Pipeline starts and checks out the intended revision
TC-02	Introduce a failing unit test	Pipeline stops before publishing the image
TC-03	Build the container image	Image build completes and can run locally
TC-04	Publish a commit-tagged image	Registry contains the expected image tag
TC-05	Deploy to Kubernetes	Rollout reaches the desired replica count
TC-06	Use an invalid image tag	Pod reports image-pull failure and rollout is not healthy
TC-07	Inspect monitoring dashboard	Relevant metrics are visible with clear units
TC-08	Roll back to previous version	Known-good application revision becomes active
TC-09	Remove registry permission	Publish stage fails safely without deploying
TC-10	Simulate an unhealthy endpoint	Readiness check prevents unready traffic

5.3 Acceptance Criteria

The system can be considered demonstrable when a valid change completes the pipeline, a failed test blocks release, an image is traceable to its source revision, Kubernetes reaches a healthy rollout, and monitoring data is visible. Recovery should be demonstrated or documented. Any result table must distinguish tests actually executed from tests that are only planned.

RESULTS, EVALUATION AND LIMITATIONS

5.4 ResultE valuation

The expected result is a repeatable path from source commit to deployment, supported by pipeline logs, image tags, Kubernetes rollout status, and monitoring dashboards. During implementation, record build duration, deployment duration, failed-stage behaviour, Pod readiness, and observed resource usage. This report does not invent benchmark figures; measurements should be collected from the actual test environment.

5.5 Manual versus Automated Evaluation

Compare the old and proposed workflows using the number of manual steps, time needed to release, number of repeated failures, traceability of the deployed version, and time required to identify a problem. A fair comparison should use similar application changes and should record the environment and measurement method. Automation may reduce routine work, but it also requires initial setup and ongoing maintenance.

5.6 Benefits

- Reduces repetitive manual deployment tasks.
- Improves traceability between source, image, pipeline run, and release.
- Provides consistent application packaging through containers.
- Supports declarative deployment and controlled rolling updates.
- Detects code and configuration problems earlier through automated checks.
- Improves operational visibility through metrics, dashboards, and logs.

5.7 Limitations

The system depends on repository availability, CI runner health, registry access, credentials, network connectivity, and cluster capacity. Monitoring cannot detect every defect, and incorrect health probes can produce misleading results. Cloud costs can increase if resources remain active. Security scanning, alerting, and automatic recovery require careful configuration and validation before production use.

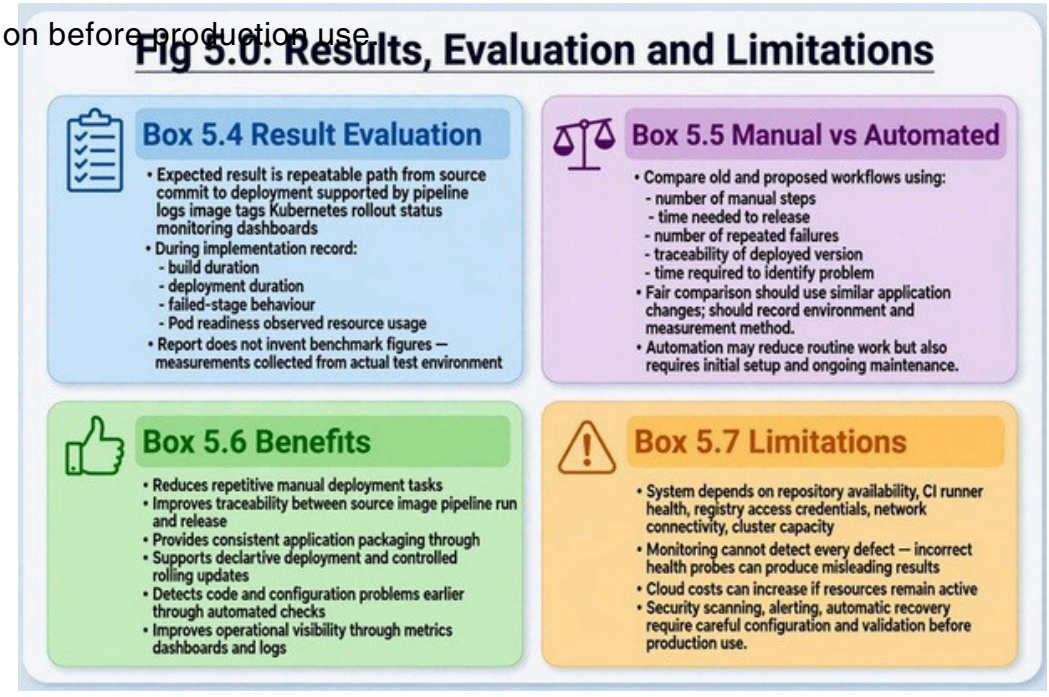


Fig 5.0: Results, Evaluation and Limitations

CONCLUSION AND FUTURE SCOPE

5.8 Conclusion

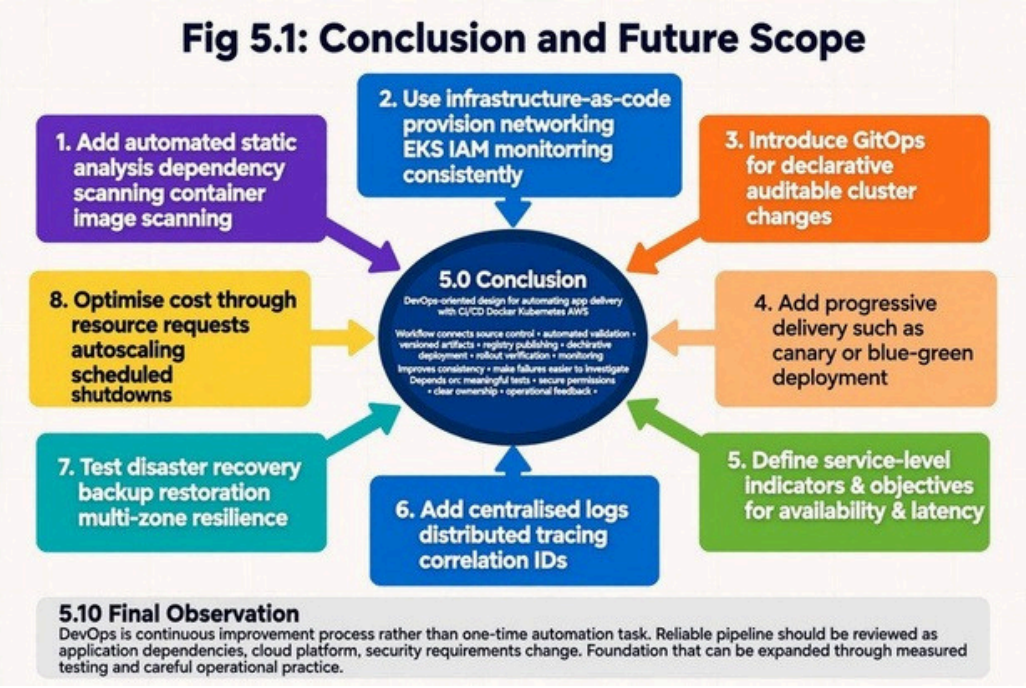
This project presents a DevOps-oriented design for automating application delivery with CI/CD, Docker, Kubernetes, and AWS. The workflow connects source control, automated validation, versioned artifacts, registry publishing, declarative deployment, rollout verification, and monitoring. This structure can improve consistency and make failures easier to investigate. The effectiveness of the system depends on meaningful tests, secure permissions, clear ownership, and operational feedback.

5.9 Future Scope

- Add automated static analysis, dependency scanning, and container image scanning.
- Use infrastructure-as-code to provision networking, EKS, IAM, and monitoring consistently.
- Introduce GitOps for declarative, auditable cluster changes.
- Add progressive delivery such as canary or blue-green deployment.
- Define service-level indicators and objectives for availability and latency.
- Add centralised logs, distributed tracing, and correlation IDs.
- Test disaster recovery, backup restoration, and multi-zone resilience.
- Optimise cost through resource requests, autoscaling, and scheduled shutdowns.

5.10 Final Observation

DevOps is a continuous improvement process rather than a one-time automation task. A reliable pipeline should be reviewed as the application, dependencies, cloud platform, and security requirements change. The project provides a foundation that can be expanded through measured testing and careful operational practice.



REFERENCES / BIBLIOGRAPHY

Thefollowingofficialdocumentation sourcesare recommended for implementation and further study. Use the version applicable to the software installed in the project, and record the date accessed when preparing the final college submission.

• Git Documentation — https://git-scm.com/doc

• Jenkins User Documentation — https://www.jenkins.io/doc/

• Docker Documentation — https://docs.docker.com/

• Kubernetes Documentation — https://kubernetes.io/docs/

• Amazon EKS User Guide — https://docs.aws.amazon.com/eks/

• Amazon ECR User Guide — https://docs.aws.amazon.com/AmazonECR/latest/userguide/

• AWS Identity and Access Management — https://docs.aws.amazon.com/iam/

• Amazon VPC Documentation — https://docs.aws.amazon.com/vpc/

• Prometheus Documentation — https://prometheus.io/docs/

• Grafana Documentation — https://grafana.com/docs/

• OWASP — https://owasp.org/

Suggested citation format

Organisation or Author. (Year or n.d.). Title of documentation page. Website or publisher. URL. Accessed on: [insert date].

Add any textbooks, journal articles, tutorials, or other sources actually used. References should be cited in the relevant chapter if the institution requires an academic citation style.

APPENDIX — IMPLEMENTATION CHECKLIST

A.1 Pre-deployment Checklist

- Source code reviewed and required tests passed.
- Container image built from the intended source revision.
- Image scanning and dependency checks completed according to project policy.
- Image pushed to the approved registry with a unique tag.
- Deployment configuration reviewed and target namespace confirmed.
- Secrets injected securely and not printed in pipeline logs.
- Readiness endpoint, ports, and resource settings verified.
- Rollout status and application health confirmed.
- Monitoring dashboard and logs checked after deployment.
- Rollback version and incident contact identified.

A.2 Useful Operational Checks

Check deployment status, list Pods and Services, inspect Pod events, review container logs, and confirm that the health endpoint is reachable. When an issue occurs, capture the time, affected version, pipeline run, error message, and relevant metrics. Avoid making several unrecorded changes at once, because they make the root cause harder to identify. **A.3 Project- specific Details to Complete**

Item	Value to fill in
Student name College	[Enter student name]
/ department Project	[Enter college and department]
guide	[Enter guide name]
AWS region	[Enter region if used]
ECR repository	[Enter repository name if used]
EKS cluster / namespace	[Enter cluster and namespace if used]
Application health endpoint	[Enter endpoint]
	[Enter dashboard reference]

End of Project Report.

APPENDIX B — PROJECT REVIEW AND VIVA PREPARATION

B.1 Key Concepts to Explain

DevOps is a collaborative approach that combines development, operations, automation, and feedback. CI validates each code change through repeatable checks. CD prepares or releases verified software through controlled stages. Docker packages an application into an image, while Kubernetes runs and manages container workloads. AWS supplies cloud infrastructure and managed services for the deployment.

B.2 Questions for Project Demonstration

1. What problem does the automated pipeline solve?
2. Why is a container image useful for deployment consistency?
3. What is the role of Amazon ECR in this design?
4. What does a Kubernetes Deployment manage?
5. How does a readiness probe protect the application?
6. What is the difference between metrics and logs?
7. Why should CI credentials follow least privilege?
8. How can a failed deployment be investigated and rolled back?

B.3 Demonstration Sequence

Begin by showing the repository and the commit that triggers the workflow. Show the pipeline stages and explain how tests gate later stages. Identify the versioned image in the registry. Show the Kubernetes Deployment and Pod readiness, then open the application health endpoint. Finally, show the monitoring dashboard and explain how logs, events, and rollback procedures help respond to a failed release.

B.4 Evidence to Attach

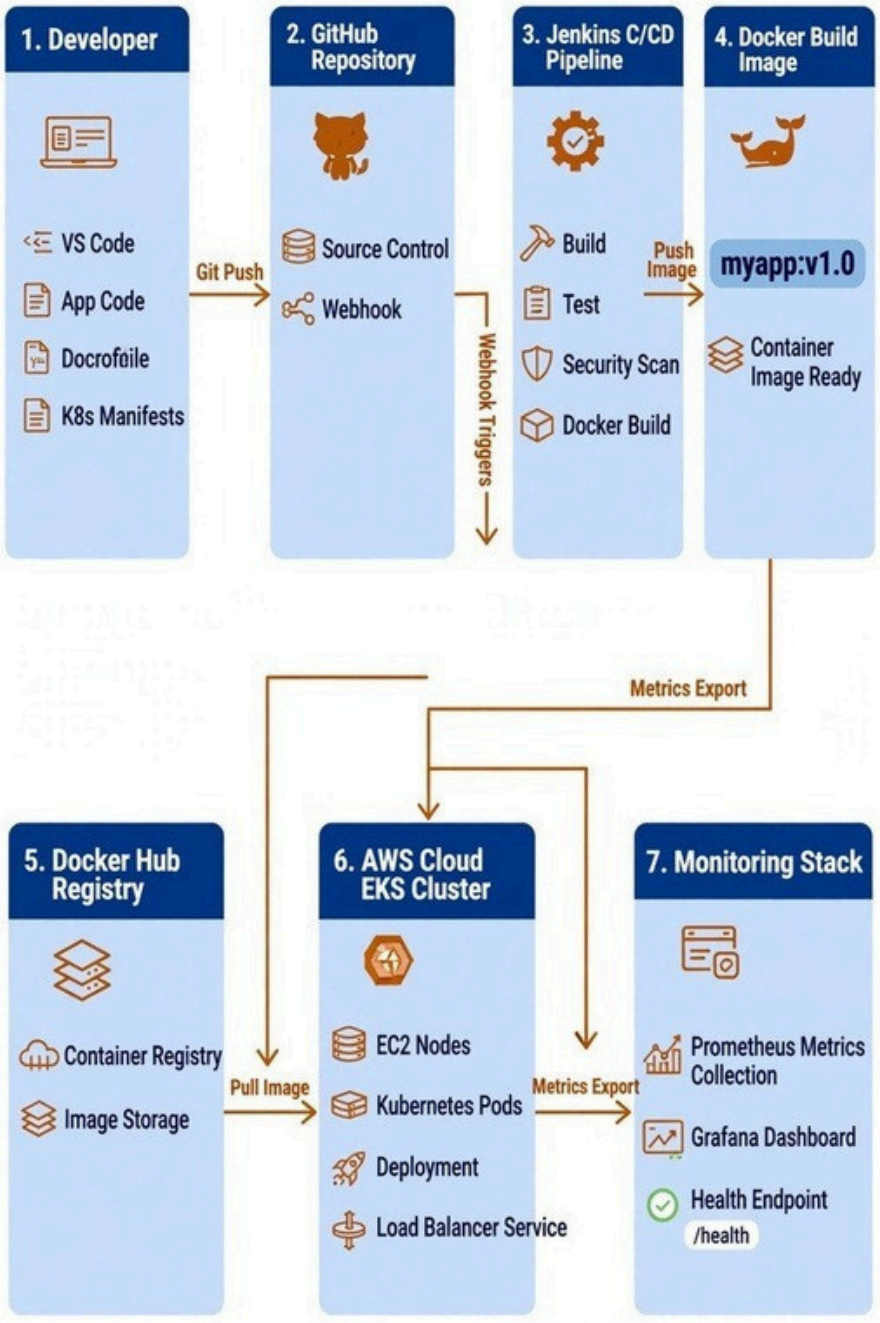
For a real implementation, attach verified screenshots of the successful pipeline, a deliberately failed test, the published image tag, the Kubernetes rollout status, the running application, and the monitoring dashboard. Add the date, environment, and brief explanation for each screenshot. Do not fabricate screenshots or performance figures; mark any planned tests as not yet executed.

B.5 Final Submission Review

Before submission, fill in the student and college details, verify chapter numbering, confirm the actual tool versions, check the reference URLs, and review the report with the project guide. Replace assumptions with observed results wherever implementation has been performed. Remove any credentials, private account details, or sensitive infrastructure information.

Proposed System Architecture - Automated Deployment & Monitoring System

CI/CD Pipeline with Docker & Kubernetes on AWS



Automated deployment flow from code commit to monitored production environment on AWS EKS